Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Rashad	rashad@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

The goals I am trying to address are to help people manage their bills that they have to pay and figure out how much they have left to use for groceries and wants.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There are a few apps like Empower and Monarch, but almost all these apps cost money.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

I want this web application to be used for figuring out how much money left you have to use for food and other things you might want. There will also be dates for your bills/reminders to not forget to pay your bills.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

Replace this text with your answer.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The user will need to input the money/budget they have, the bills they need to pay, and when they are due.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The processing will compose of subtracting the total bills from the money/budget you have. It will also list dates per month of what is due.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get the money they will have left after paying all their bills and any reminders that show what bills they have left to pay with the due date.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Bills, Budget, Date, History

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

The details I will need are:
Money/Budget
Bill name, due date, price

Pressing paid will subtract from the budget and delete the bill but will add it to your history.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The most complex part seems to be the reminders of the bills because it will have things like a PAID/UNPAID button. Also, creating bills visually might take some time.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

If I use an LLM I will ask about how the code works in the background and what each part does before typing it myself to help me connect the pieces.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I want to build something that people can use to help them with their bills and I might try to make some nice UI.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to create a simple web tool that can help people manage their bills and not to forget to pay them. What are 3 common problems people face when paying bills, and what are 3 simple features a basic tool should have.

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The LLM talked about a bill due date tracker, payment status tracker, and a monthly bill summary.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Me: A monthly bill summary might be a bit complicated. What could be a simpler alternative?

A simpler alternative would be 3. Upcoming Bills List.

Instead of calculating monthly totals or creating charts, the tool simply displays the user's next bills and when they are due.

3. Upcoming Bills List

Problem it solves: People forget which bills are coming up and when they need to pay them.

How it works:

- *Display bills sorted by due date.*
- *Show each bill's name, amount, and due date.*

- *Highlight bills that are due soon or overdue.*

Example:

Upcoming Bills

Electricity

$120 · Due November 3

Due soon

Internet

$80 · Due November 8

Upcoming

Rent

$900 · Due November 10

Upcoming

Your three main features would now be:

1. *Bill Due-Date Tracker: Add bills with names, amounts, and due dates.*
2. *Payment Status Tracker: Mark bills as paid or unpaid.*
3. *Upcoming Bills List: See which bills are coming up, due soon, or overdue.*

This is much easier to implement in Drafter because it mainly requires storing bill information, comparing dates, and displaying a sorted list.

I can also help turn these three features into a simple project proposal for your CS1 final project.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

It has become more focused toward showing the user what they have to pay, when they have to pay, and if it was paid or not.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

The resources I will use are the drafter documentation and the LLM.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

My step by step process would be to read the code myself and to see what went wrong. I might ask the LLM what it is trying to do and the thought process behind what it did.

Part 7: LLM Conversation

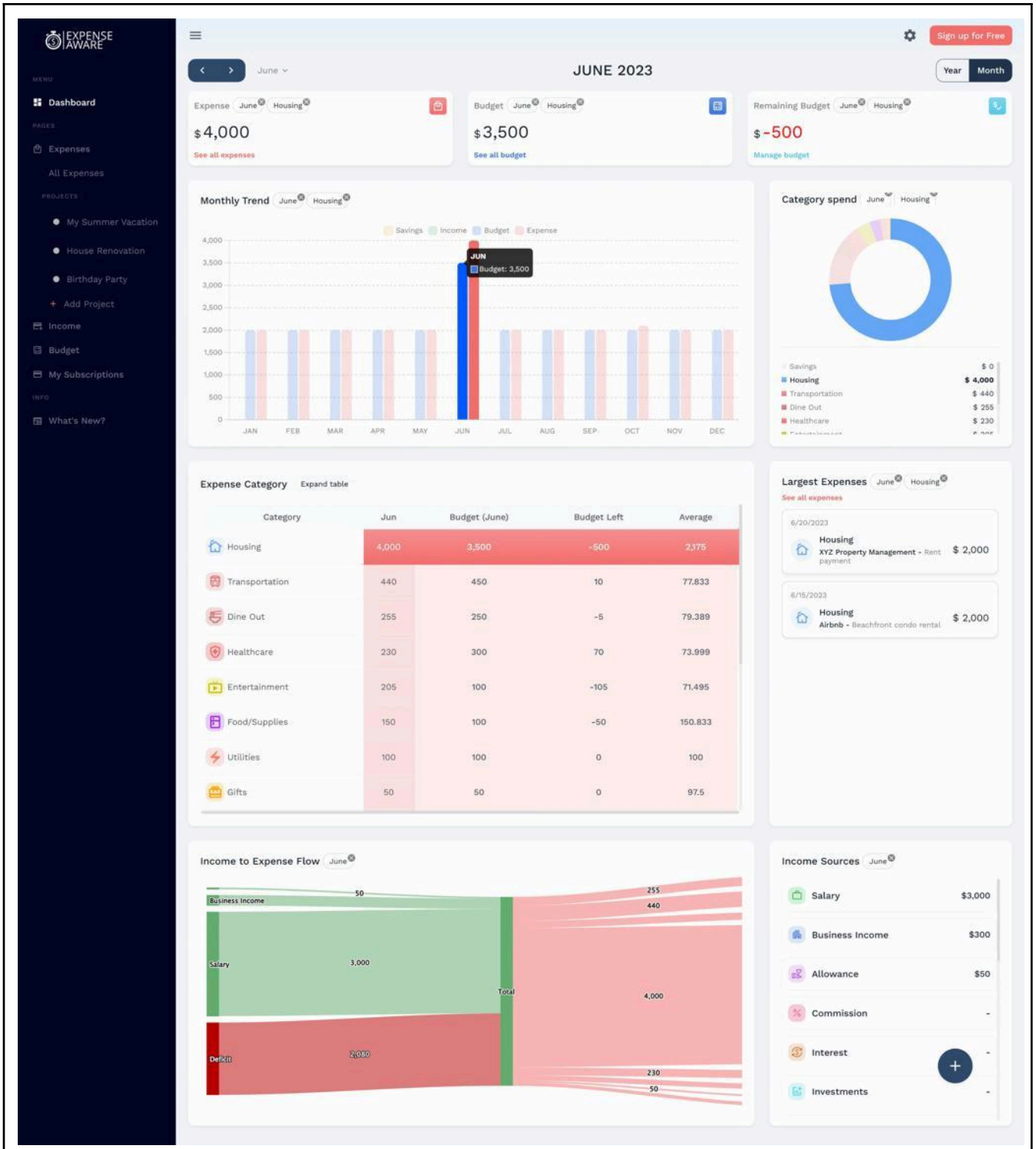
If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Me: Give me ideas for this drafter project about budgeting

Budgeting is a great direction for your Drafter project because it solves a real, recurring problem and lets you build a professional-looking app with useful calculations, charts, and saved data — all without a backend or database.

Rather than making just a basic expense tracker, I'd recommend choosing one specific budgeting problem and building an app that helps people solve it.

8 budgeting project ideas



1. BudgetWise — Personal Budget Planner

Best overall

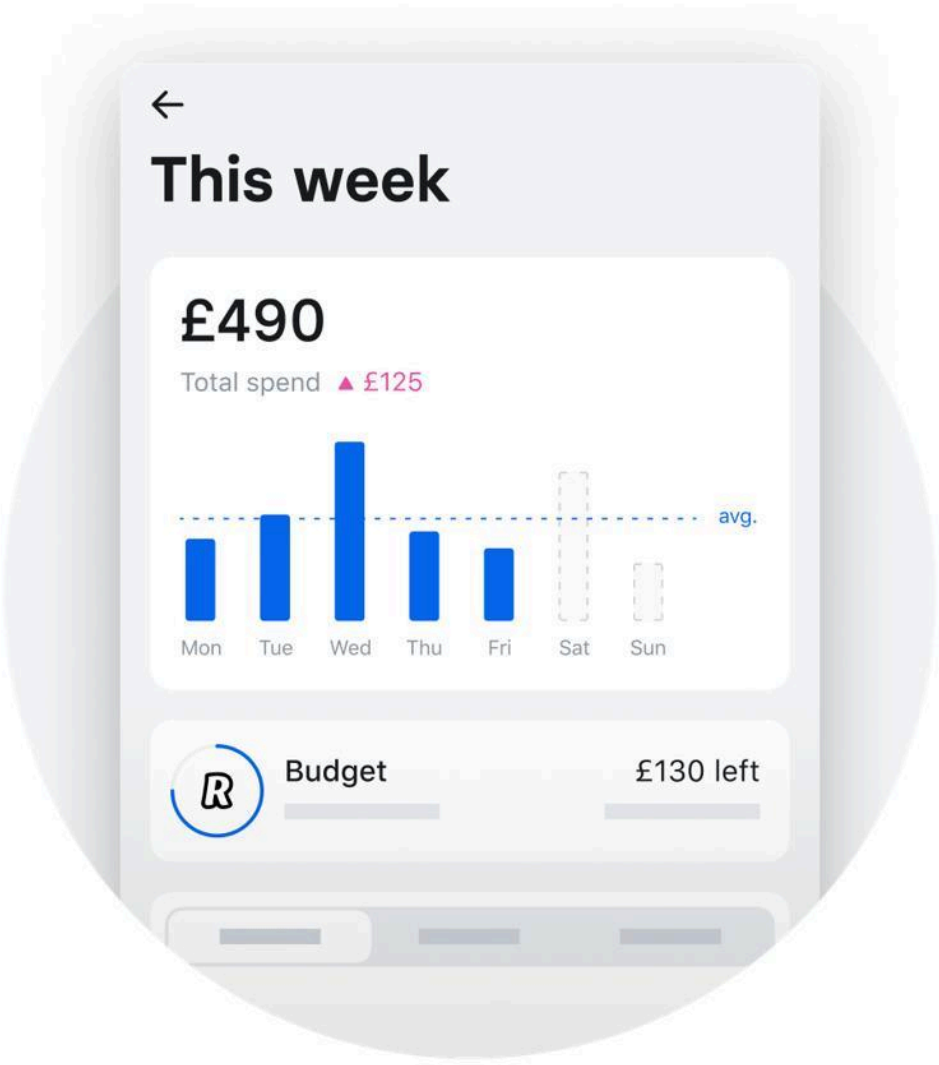
Problem: People don't know how much money they can safely spend after bills and

necessities.

Features

- Enter monthly income and fixed expenses.
- Set spending limits for food, shopping, transportation, and entertainment.
- See remaining money and spending breakdowns.
- Compare planned spending with actual spending.

Difficulty: Easy–moderate



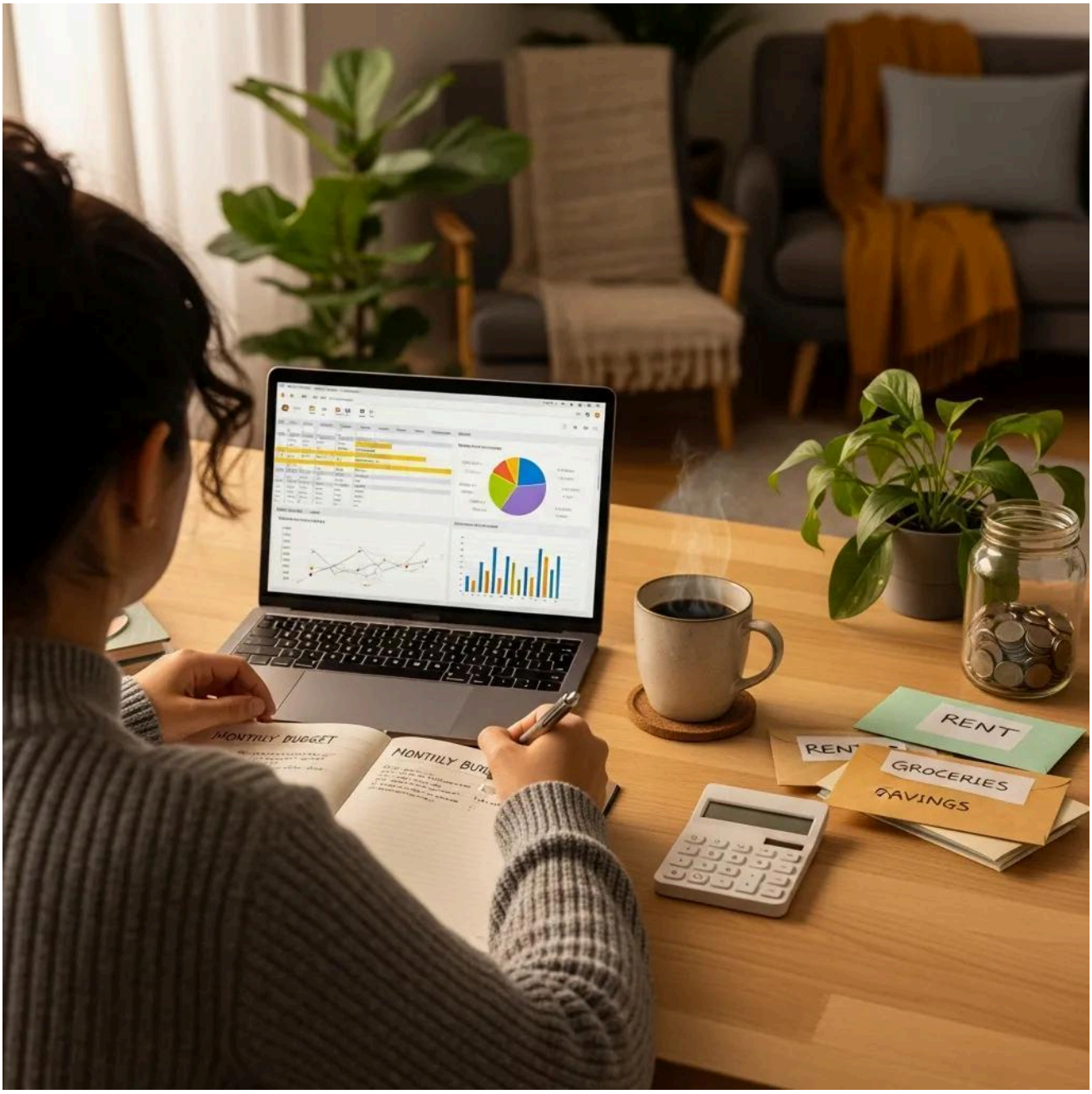
2. SpendSmart — How Much Can I Spend Today?

Problem: People spend too much early in the month and run short before their next paycheck.

Features

- Enter available money and the date of the next paycheck.
- Subtract upcoming bills and planned savings.
- Calculate a suggested daily or weekly spending allowance.
- Recalculate the allowance whenever a new expense is added.

Difficulty: Easy–moderate · Very practical



3. StudentFunds — College Student Budgeting

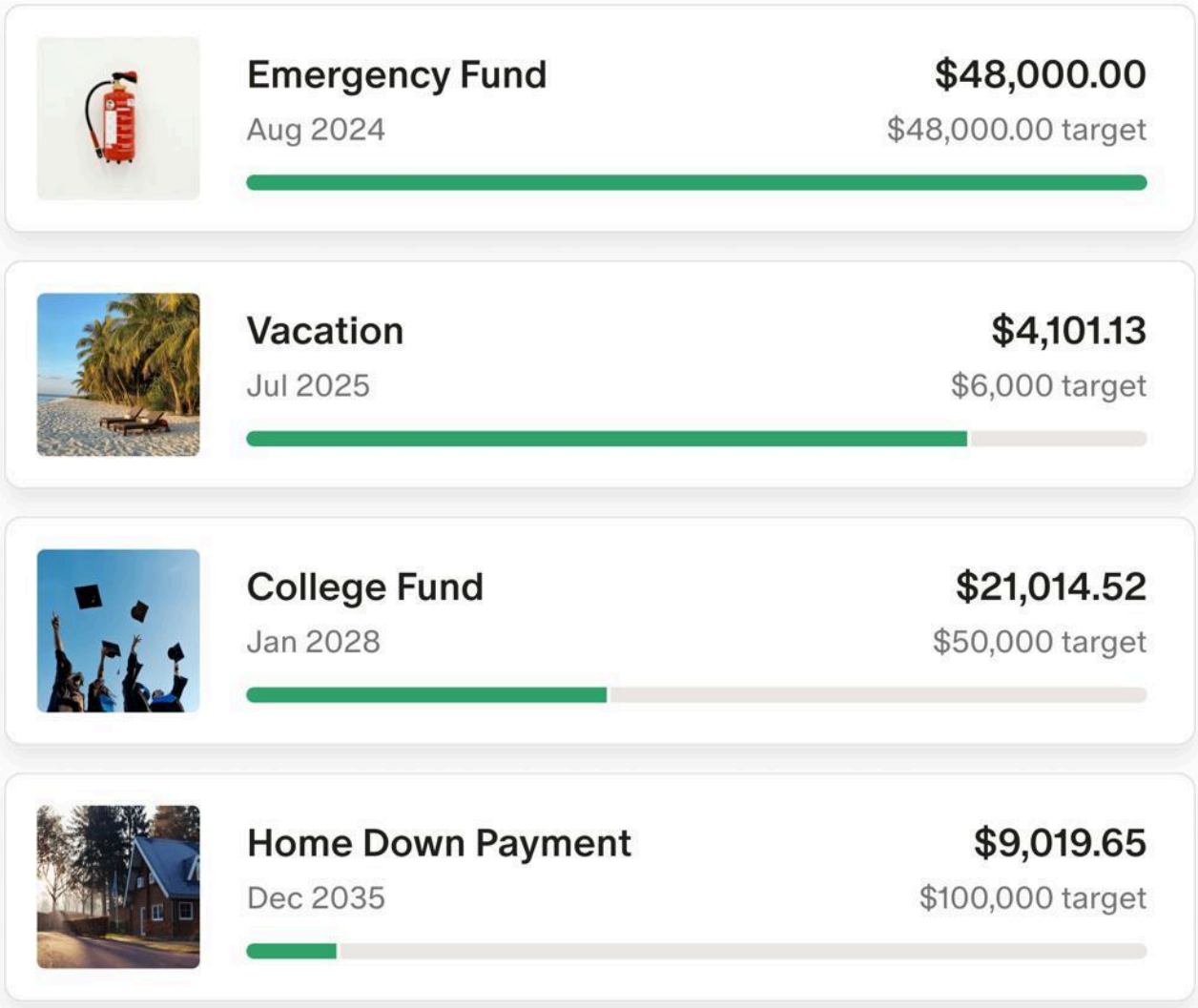
Problem: Students have limited income and need to balance food, transportation, school supplies, social activities, and bills.

Features

- Track money from jobs, allowances, or financial aid.

- Separate needs from wants.
- Estimate how long available money will last.
- Compare spending across weeks or months.

Difficulty: Moderate · A clearly defined target audience



4. GoalFund — Saving for Something Important

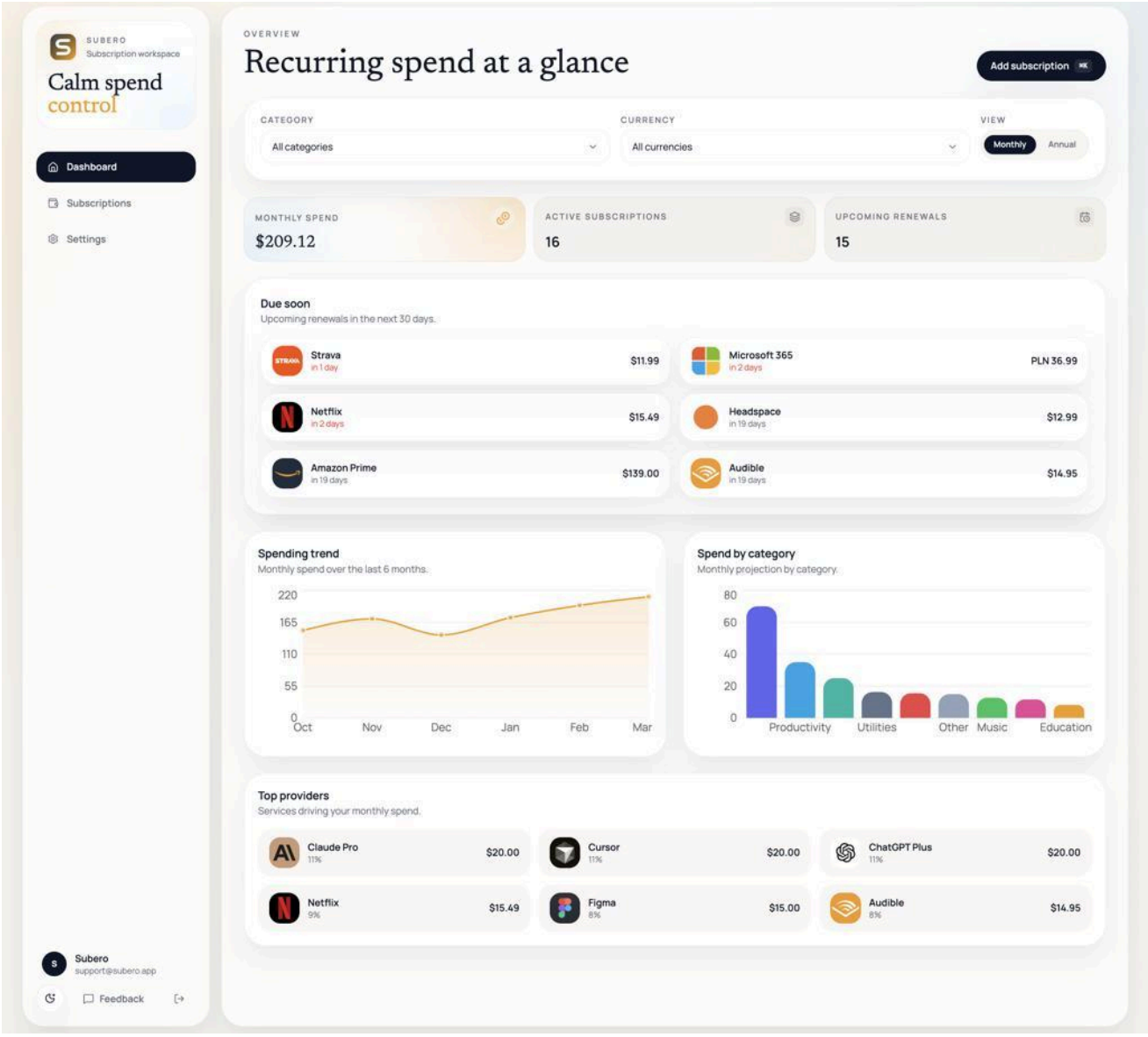
Problem: People want to save for a car, vacation, emergency fund, or another goal but don't know how much to save regularly.

Features

- Set a target amount and target date.

- Record contributions toward the goal.
- Calculate required weekly or monthly savings.
- Display progress and estimated completion dates.

Difficulty: Easy–moderate



5. SubWatch — Subscription Cost Tracker

Problem: People forget about recurring payments and underestimate how much subscriptions cost over time.

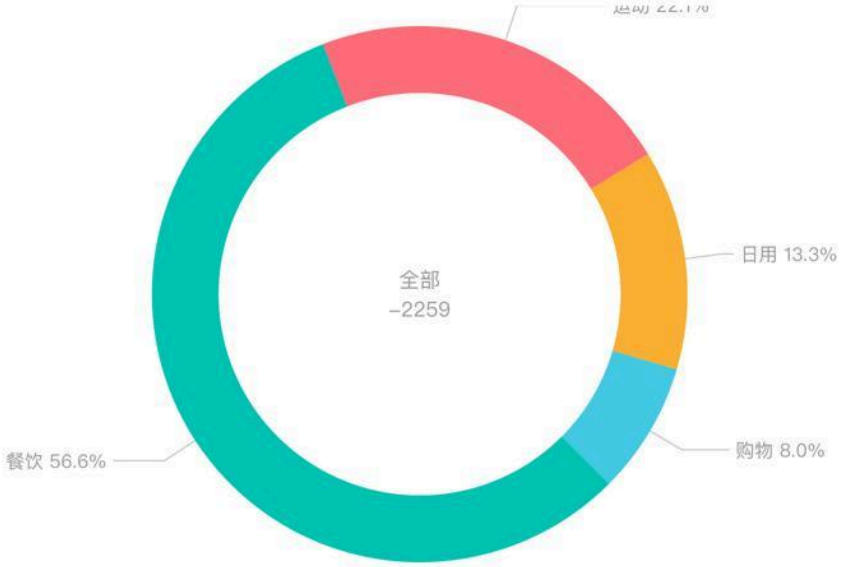
Features

- Record subscriptions, prices, and billing frequencies.
- Calculate monthly and yearly costs.
- Identify the most expensive subscriptions.
- Estimate savings if selected subscriptions are cancelled.

Difficulty: Easy · Simple but useful

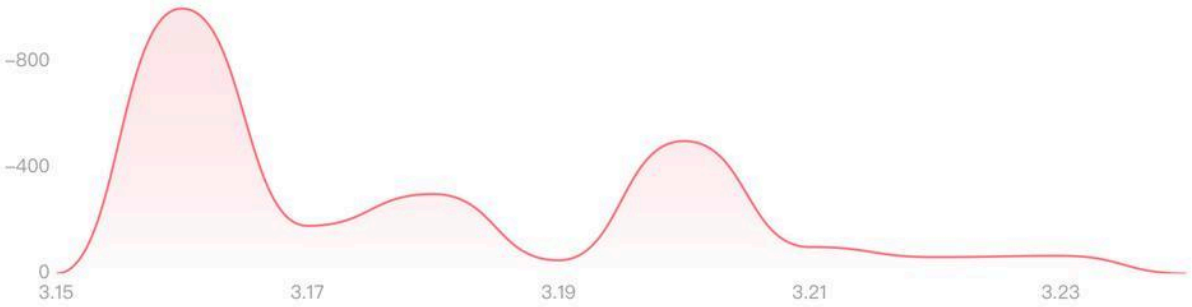
分类统计

-¥2259 支出 • 3月 • 一级分类



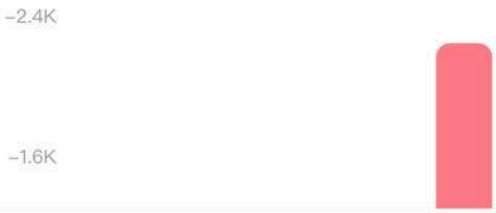
日趋势

-¥2259 支出 • 3月



月度收支

-¥2259 支出 • 2022年



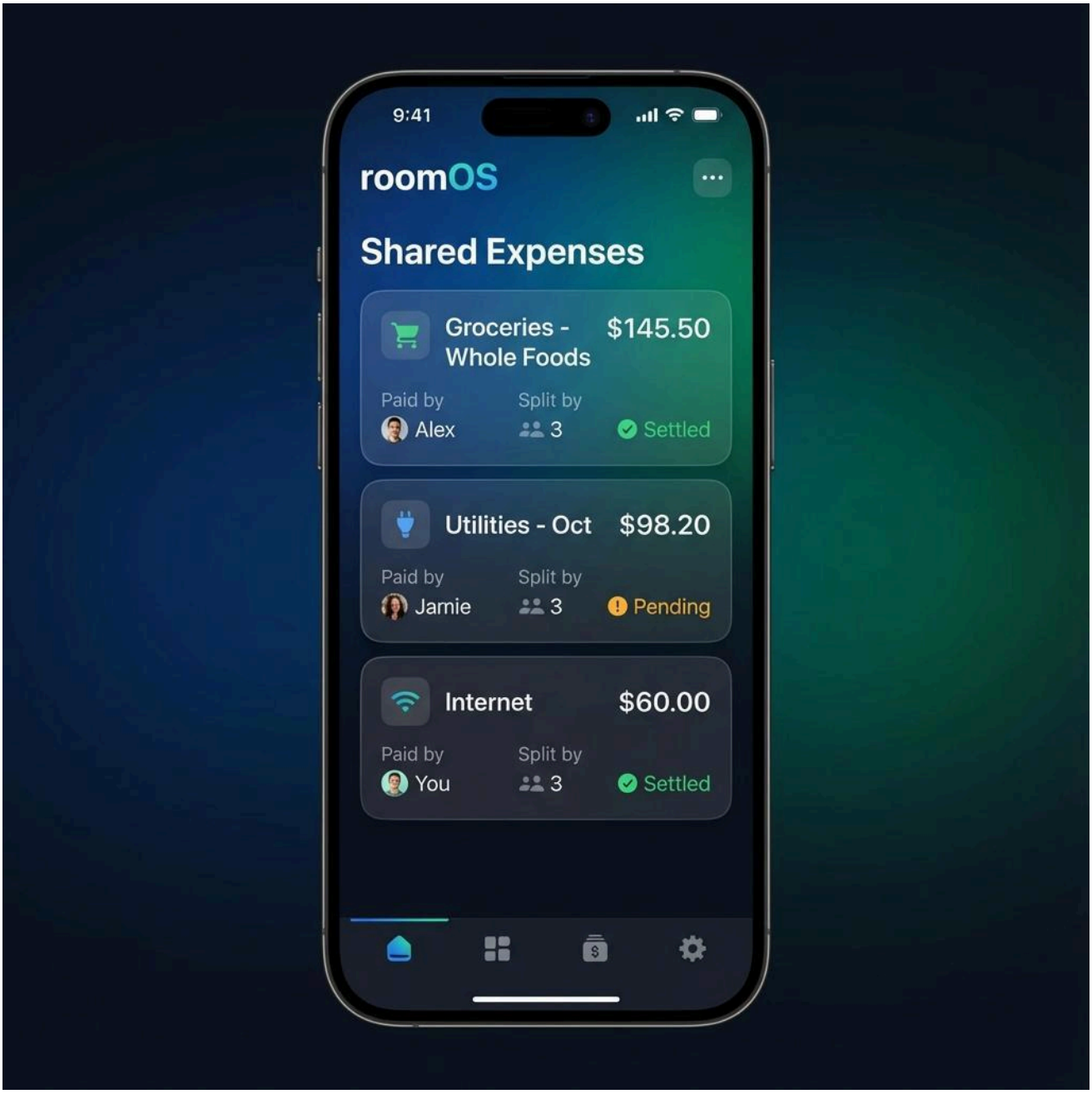
6. MoneyMap — Spending Pattern Analyzer

Problem: People know they are spending money but don't understand where it goes.

Features

- Enter transactions or upload a CSV of expenses.
- Automatically categorize transactions using simple keyword rules.
- Generate charts of spending by category.
- Compare months and identify the largest increases.

Difficulty: Moderate · Strong data-processing component



7. SplitFair — Shared Expense Calculator

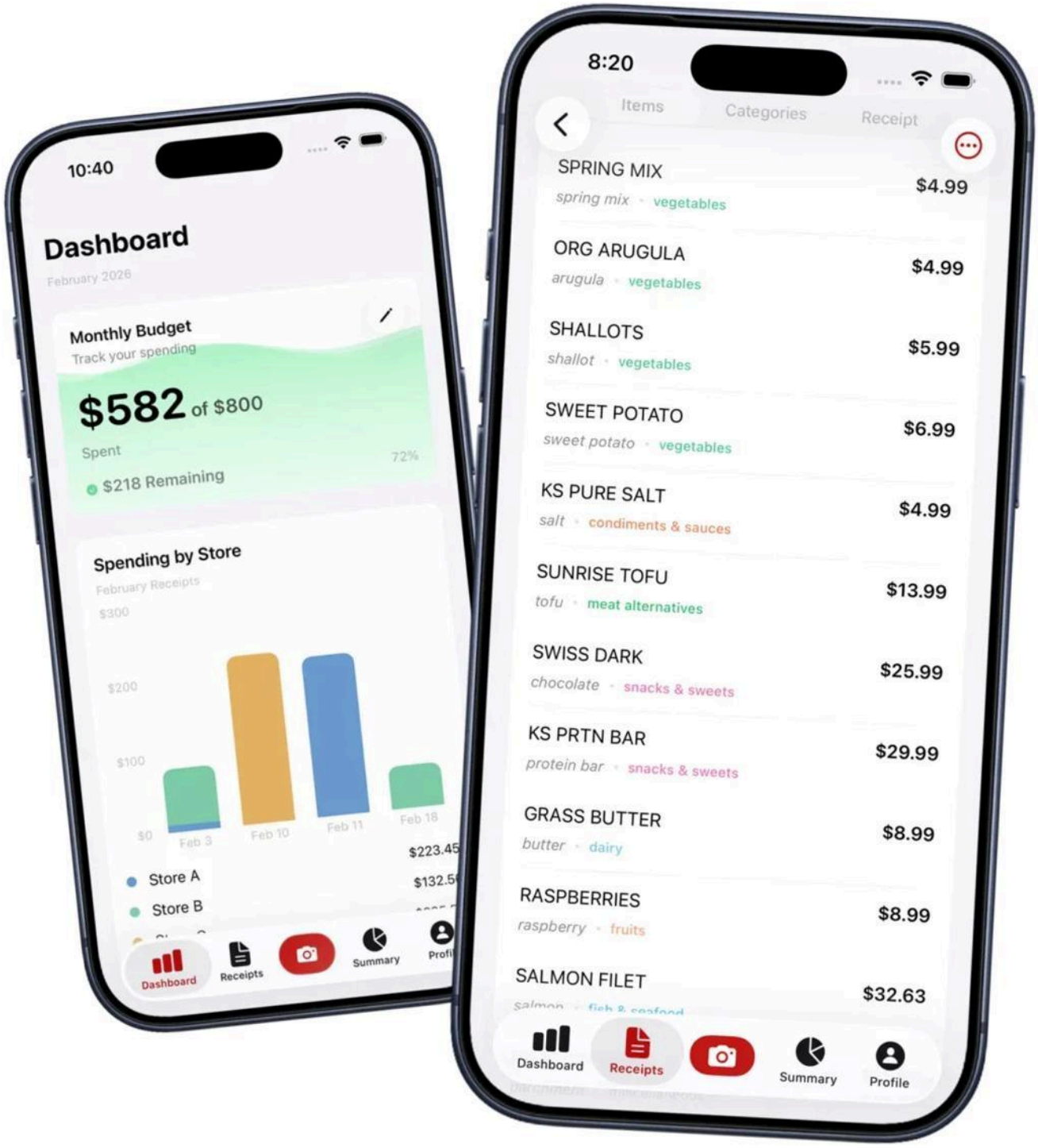
Problem: Roommates, friends, and couples struggle to divide shared expenses fairly.

Features

- Record shared purchases and who paid.
- Support equal or custom expense splits.

- Calculate who owes whom.
- Keep a running balance of shared expenses.

Difficulty: Moderate · Good algorithmic challenge



8. ShopWithin — Shopping Budget Assistant

Problem: People go shopping without knowing whether their planned purchases fit their budget.

Features

- Enter a shopping budget and a list of items with prices.
- Calculate the total and remaining money.
- Flag when the list exceeds the budget.
- Let users compare alternative items and prioritize necessities.

Difficulty: Easy–moderate · Great for an interactive interface

Which would make the strongest final project?

Project	Main strength	Difficulty
BudgetWise	Complete budgeting workflow	Easy–moderate
SpendSmart	Solves the immediate question, “Can I afford this?”	Easy–moderate
StudentFunds	Specific audience and realistic financial constraints	Moderate
MoneyMap	Charts, CSV uploads, and data analysis	Moderate
SplitFair	Interesting calculations and expense-splitting logic	Moderate

My recommendation: SpendSmart

I'd choose SpendSmart — How Much Can I Spend Today? because it solves a specific problem rather than simply storing transactions.

Example of how your app could work

Illustrative example with sample numbers

Available money

$800

Bills before payday

$300

Planned savings

$100

Available to spend

$400

If 10 days remain until payday, the app calculates a suggested allowance of $40 per day. When the user logs a $25 purchase, it updates the remaining balance and spending allowance.

Features you could realistically finish in three weeks:

- Add and delete expenses.
- Set income, bills, and savings targets.
- Calculate daily and weekly spending limits.
- Display a spending chart.
- Save everything in browser local storage.

Drafter is well suited to this because you can implement the calculations in Python, use charts for spending breakdowns, and save the user's data locally without setting up a server.

One important design decision: the app should describe its spending allowance as a planning estimate, not a guarantee that the user can afford every purchase.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.